

39 -- AI Provider Routing & 9Router Integration

What it is

When your application needs AI capabilities (text generation, embeddings), it talks to an AI provider -- a company running powerful models on their servers. Examples include Google Gemini, Groq, and OpenRouter. Each provider has its own API endpoint, its own pricing, and its own rate limits.

An **AI router** (sometimes called an AI proxy or gateway) is a piece of software that sits **between** your application and all those providers. Instead of your app calling Gemini directly, it calls the router. The router then decides which actual provider to forward the request to, based on rules you configure: cost, availability, speed, or simple priority ordering.

Think of it as adding a middleman whose entire job is making sure your request always gets answered, even if one provider is down or rate-limiting you.

The 3-tier fallback concept

9Router (the specific router we integrated) organises providers into three tiers:

```
Tier 1: Subscription providers (you pay monthly)
  -- Always tried first, lowest latency, most reliable.
  -- Example: your paid OpenAI or Anthropic key.

Tier 2: Cheap / pay-per-token providers
  -- Tried if Tier 1 fails or is unavailable.
  -- Example: Together AI, Fireworks, etc.

Tier 3: Free-tier providers
  -- Last resort, may have strict rate limits.
  -- Example: free-tier Groq, free-tier OpenRouter models.
```

When your app sends a request through 9Router, it walks down the tiers: try Tier 1 first, and if that fails (timeout, rate limit, error), automatically try Tier 2, then Tier 3. Your application code never sees the fallback -- it just gets a successful response (or an error only if ALL tiers failed).

OpenAI-compatible endpoint

The key technical detail is that 9Router exposes an **OpenAI-compatible API**. That means any code already written to talk to OpenAI's `/v1/chat/completions` or `/v1/embeddings` endpoints works with 9Router by just changing the base URL and API key. No new SDK, no new request format.

Real-life analogy

Imagine you need to fly from Budapest to London tomorrow. You could:

1. **Call each airline yourself** -- check Ryanair, then Wizz Air, then British Airways, then Lufthansa. If one is sold out, you call the next. If one's website is down, you try another. This is exhausting.
2. **Use a travel agent** -- you tell them "I need Budapest to London tomorrow" and they check all airlines for you. If Ryanair is sold out, they automatically check Wizz Air. If Wizz Air's system is down, they try British Airways. You just get a ticket.

The travel agent is the router. You (the application) make one request. The travel agent (9Router) checks multiple airlines (AI providers) in priority order and hands you back a booking (an AI response). If your preferred airline (provider) is cancelled (rate-limited), the agent rebooks you (falls back to another provider) -- all invisible to you.

In our project: - **You** = the ETL pipeline or Search API - **Travel agent** = 9Router running locally on port 20128 - **Airlines** = Gemini, Groq, OpenRouter, OpenAI, Anthropic, and 60+ others - **Ticket** = the generated text or embedding vector

Why we used it here

Our project kept hitting **OpenRouter free-tier rate limits** -- HTTP 429 ("Too Many Requests") errors. During a full ETL run, we make hundreds of LLM calls (translating company descriptions, generating enrichments) and hundreds of embedding calls. Free-tier providers only allow a handful of requests per minute, so the pipeline would stall, retry, stall again, and sometimes fail entirely.

We already had retry logic with exponential backoff (learning doc #25), but retrying the same rate-limited provider just wastes time. What we really needed was **automatic failover to a different provider**.

9Router solves this in one integration: - It runs locally (no cloud dependency for the routing logic itself). - It presents a single OpenAI-compatible endpoint. - Behind the scenes, it has access to 60+ providers and routes across them with 3-

tier fallback. - If our primary provider is rate-limited, 9Router silently retries on another provider.

This ties directly to two project requirements: - **R22 (Multi-provider LLM)** -- the admin can select 9Router as the active LLM provider alongside Gemini, Groq, and OpenRouter. - **R23 (Multi-provider Embeddings)** -- the admin can select 9Router as the active embedding provider alongside Gemini, Jina, and local BGE-M3.

Code walkthrough

1. NineRouterLLM class (`bilingual_etl/providers/llm_provider.py`)

This is the LLM provider class. It follows the exact same pattern as the existing `GroqLLM` and `OpenRouterLLM` classes -- inheriting from `LLMProvider` and implementing `generate()`.

```
class NineRouterLLM(LLMProvider):
    """9Router -- local AI router with 3-tier fallback across 60+ providers."""

    name = "9router"

    def __init__(self):
        self.api_key = os.getenv("NINE_ROUTER_API_KEY", "")
        if not self.api_key:
            raise ValueError("NINE_ROUTER_API_KEY not set in environment")
        base_url = os.getenv("NINE_ROUTER_URL", "http://localhost:20128")
        self.url = f"{base_url}/v1/chat/completions"
        self.model = os.getenv("NINE_ROUTER_LLM_MODEL", "kr/claude-sonnet-4.5")

    def generate(self, prompt: str, system_prompt: str = "") -> str:
        messages = []
        if system_prompt:
            messages.append({"role": "system", "content": system_prompt})
        messages.append({"role": "user", "content": prompt})

        payload = {"model": self.model, "messages": messages, "temperature": 0.3}
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json",
        }

        resp = requests.post(self.url, json=payload, headers=headers, timeout=120)
        resp.raise_for_status()
        data = resp.json()

        text = data["choices"][0]["message"]["content"]
        logger.info(f"LLM call served by provider=9router model={self.model}")
        return text.strip()
```

Line-by-line breakdown:

- `name = "9router"` -- This string is what appears in the `app_config` DB table and in log lines. It must match the key in the `PROVIDERS` dictionary at the bottom of the file.
- `__init__` reads three environment variables:
- `NINE_ROUTER_API_KEY` -- authentication token for 9Router (required).
- `NINE_ROUTER_URL` -- where 9Router is running. Defaults to `http://localhost:20128`, which is 9Router's standard local port.
- `NINE_ROUTER_LLM_MODEL` -- which model to request. Defaults to `kr/claude-sonnet-4.5`. The `kr/` prefix is a 9Router convention for routing through its provider tiers.
- `self.url` appends `/v1/chat/completions` to the base URL -- this is the standard OpenAI-compatible chat endpoint.
- `generate()` builds the same `messages` array format that OpenAI, Groq, and OpenRouter all use. Because 9Router speaks the OpenAI protocol, the request body is identical.
- `timeout=120` -- doubled from the 60-second timeout used by other providers, because 9Router may need extra time to try fallback providers.
- The logger line (`provider=9router model=...`) satisfies the UAT evidence requirement that every LLM call logs which provider served it.

Registration in the provider dictionary:

```
PROVIDERS = {  
    "gemini": GeminiLLM,  
    "groq": GroqLLM,  
    "openrouter": OpenRouterLLM,  
    "9router": NineRouterLLM,      # <-- new entry  
}
```

This single dictionary is what `get_llm_provider()` looks up when the admin selects a provider in the UI. Adding a new provider to the system is: write the class, add it here.

2. NineRouterEmbedding class (`bilingual_etl/providers/embedding_provider.py`)

The embedding counterpart follows the same pattern, but calls the `/v1/embeddings` endpoint instead:

```

class NineRouterEmbedding(EmbeddingProvider):
    """9Router -- local AI router proxying OpenAI-compatible embedding endpoints."""

    name = "9router"
    dimensions = 768

    def __init__(self):
        self.api_key = os.getenv("NINE_ROUTER_API_KEY", "")
        if not self.api_key:
            raise ValueError("NINE_ROUTER_API_KEY not set in environment")
        base_url = os.getenv("NINE_ROUTER_URL", "http://localhost:20128")
        self.url = f"{base_url}/v1/embeddings"
        self.model = os.getenv("NINE_ROUTER_EMBED_MODEL", "text-embedding-3-small")

    def embed(self, texts: list[str]) -> list[list[float]]:
        payload = {"model": self.model, "input": texts}
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json",
        }

        resp = requests.post(self.url, json=payload, headers=headers, timeout=60)
        resp.raise_for_status()
        data = resp.json()

        results = [item["embedding"] for item in data["data"]]
        logger.info(
            f"Embedded {len(texts)} text(s) via provider=9router "
            f"model={self.model} dims={self.dimensions}"
        )
        return results

```

Key differences from the LLM class:

- Endpoint is `/v1/embeddings` (not `/v1/chat/completions`).
- Uses `NINE_ROUTER_EMBED_MODEL` env var, defaulting to `text-embedding-3-small`.
- `dimensions = 768` -- matches the project's default vector column width. If you change this, the pgvector columns must also be resized.
- `embed()` accepts a list of texts and returns a list of float-lists -- matching the `EmbeddingProvider` base interface.
- It sends all texts in a single batch (the `input` field), unlike the Gemini provider which loops one-by-one. This is more efficient.

Registration:

```

PROVIDERS = {
    "gemini": GeminiEmbedding,
    "jina": JinaEmbedding,
    "local_bge_m3": LocalBGEEmbedding,
    "9router": NineRouterEmbedding,  # <-- new entry
}

```

3. Admin config availability checks (`search_api/routers/admin_config.py`)

When an admin tries to activate a provider, the API must verify that the required environment variable is set. Otherwise, the ETL would fail at runtime. Two places were updated:

Validation on save (`_check_provider_available`):

```
if key == "llm_provider":
    required_keys = {
        "groq": "GROQ_API_KEY",
        "openrouter": "OPENROUTER_API_KEY",
        "gemini": "GEMINI_API_KEY",
        "9router": "NINE_ROUTER_API_KEY",      # <-- added
    }
```

If someone selects 9Router but has not set `NINE_ROUTER_API_KEY` in the `.env` file, the API returns HTTP 400 with a clear error message instead of silently saving a broken configuration.

For embeddings, a similar check was added:

```
if active == "9router" and not os.getenv("NINE_ROUTER_API_KEY"):
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="NINE_ROUTER_API_KEY is not set",
    )
```

Availability endpoint (`GET /admin/config/providers/availability`):

This endpoint returns a list of all providers and whether they are currently usable. The Admin UI calls this on page load to grey out unavailable options.

```
llm_providers = {
    "groq": ("GROQ_API_KEY", None),
    "openrouter": ("OPENROUTER_API_KEY", None),
    "gemini": ("GEMINI_API_KEY", None),
    "9router": ("NINE_ROUTER_API_KEY", None),    # <-- added
}
```

And in the embedding section:

```
emb_providers = [
    ("gemini", lambda: bool(os.getenv("GEMINI_API_KEY")), "GEMINI_API_KEY not set"),
    ("jina", lambda: bool(os.getenv("JINA_API_KEY")), "JINA_API_KEY not set"),
    ("9router", lambda: bool(os.getenv("NINE_ROUTER_API_KEY")), "NINE_ROUTER_API_KEY not set"),
]
```

4. Admin UI (admin-ui/src/pages/ProviderSettings.jsx)

The React component that renders the provider selection buttons only needed two lines changed -- adding "9router" to the option arrays:

```
const LLM_OPTIONS = ["groq", "openrouter", "gemini", "9router"];
const EMBEDDING_OPTIONS = ["gemini", "jina", "local_bge_m3", "9router"];
```

The rest of the component already handles: - Fetching availability from the API and greying out unavailable options. - Calling the save endpoint when a button is clicked. - Showing success/error messages.

This is a direct benefit of the **provider abstraction pattern**: adding a new provider to the UI is just appending a string to an array, because the component treats all providers identically.

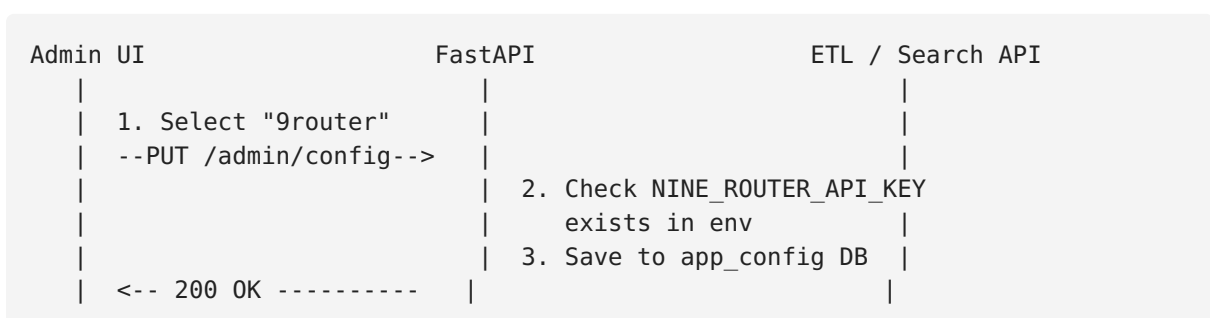
5. Environment variables

Five env vars control the 9Router integration:

Variable	Required?	Default	Purpose
NINE_ROUTER_API_KEY	Yes (if using 9router)	--	Auth token for the 9Router instance
NINE_ROUTER_URL	No	http://localhost:20128	Base URL where 9Router is running
NINE_ROUTER_LLM_MODEL	No	kr/claude-sonnet-4.5	Model identifier for text generation
NINE_ROUTER_EMBED_MODEL	No	text-embedding-3-small	Model identifier for embeddings

Add these to your .env file when you want to use 9Router. If the API key is not set, the provider will appear greyed out in the Admin UI and cannot be activated.

How the pieces connect (request flow)



```
4. Next ETL run starts  
   reads app_config  
   -> active = "9router"  
  
5. get_llm_provider()  
   -> NineRouterLLM()  
  
6. generate_with_retry()  
   -> POST to 9Router  
       localhost:20128  
  
7. 9Router forwards to  
   best available  
   provider (Tier 1-3)
```

How to explain this in an interview / to a teammate

"Our ETL pipeline calls AI providers for translation and embedding generation, but free-tier providers have strict rate limits that cause failures during large runs. Instead of hardcoding fallback logic for each provider, we integrated 9Router -- a local AI proxy that exposes a single OpenAI-compatible endpoint. Behind the scenes, it has access to 60+ providers and automatically falls back across three tiers: subscription, cheap, and free. Because it speaks the OpenAI protocol, integrating it was just adding a new provider class to our existing abstraction layer -- same interface, same logging, same retry logic. The admin can switch to it (or away from it) in the UI without any code changes or redeployment."

Key takeaways

1. **Provider abstraction pays off** -- adding 9Router required one new class per module (LLM + embedding), one dictionary entry each, one string in the UI array, and a few env-var checks. No changes to the ETL pipeline logic, the search API, or the retry system.
2. **OpenAI-compatible APIs are a de facto standard** -- many AI tools (routers, proxies, local model servers) deliberately implement the same request/response format as OpenAI's API. This means code written for one provider often works with another just by changing the URL.
3. **Routing is different from retrying** -- our existing retry logic (doc #25) retries the same provider after a backoff. Routing sends the request to a different provider entirely. Both are useful; they complement each other.
4. **Environment-gated activation** -- the API refuses to activate a provider whose API key is missing. This "fail early" check prevents a broken configuration from reaching the ETL, where debugging would be harder.